Benchmarking Local LLM Inference Servers Under Concurrent Load: A Server-Centric Testing Methodology

Bharath Thiruveedula
The University of Texas at Austin
Austin, Texas, USA
bt24326@my.utexas.edu

Kaitlin Arnold
The University of Texas at Austin
Austin, Texas, USA
kma4239@my.utexas.edu

Ronald Joseph Marcelin Franklin
The University of Texas at Austin
Austin, Texas, USA
rm57746@my.utexas.edu

Abstract

This paper presents a server-centric testing methodology for OpenAI-compatible local large-language-model (LLM) inference backends, applied to vLLM, Ollama, and llama-cpp-python. On top of standard end-to-end latency, time-to-first-token (TTFT), throughput, and JSON validity measurements, we layer eight complementary testing methodologies—contract conformance, streaming protocol compliance, parameter honoring, determinism, statistical performance regression, differential/N-version testing, concurrency isolation, and negative error-path testing—drawing on established testing conventions from projects such as vLLM, llama.cpp, and TGI. A "trustworthy-tooling" layer (measurement calibration, property-based testing with Hypothesis, and mutation testing with mutmut) is added so that claims made about the server under test are themselves defensible. The unit-level suite comprises 184 passing tests, overall line coverage stands at 75%, and mutation testing of the statistical-regression module kills 247 of 281 generated mutants (88%).

Keywords

LLM inference, benchmarking, software testing, contract testing, metamorphic testing, property-based testing, mutation testing, vLLM, llama.cpp, Ollama

1 Introduction

Benchmarks of LLM inference servers (vLLM, Ollama, llama-cpp-python) typically treat the server as a black box whose output is sampled and aggregated into latency and throughput numbers. For a software testing course this is insufficient: a single run can yield any numbers, and a single-run comparison can be dominated by variance and cannot separate an honest regression from noise, a mis-implemented streaming parser from a mis-implemented server, or a broken schema from broken text.

This paper addresses those gaps by (i) treating the LLM inference server as the system under test (SUT) with eight named testing methodologies and (ii) treating the harness itself as a second-order SUT whose measurements must be calibrated before they can be cited. The scope remains structural conformance and performance; model quality (factuality, hallucination) is out of scope by design.

2 Threats Addressed

The revised methodology targets four threats that went unaddressed in the original report:

- **Measurement noise.** Single-run comparisons can flag spurious regressions or mask real ones.
- **Parser fragility.** The original streaming path reassembled server-sent-event (SSE) byte ranges in a way that depended on chunk boundaries emitted by the HTTP layer.
- **Silent protocol drift.** A backend can return plausible-looking text while violating the OpenAI response envelope (missing fields, wrong `finish_reason`, unstable `id` across chunks).
- **Unverified tooling.** Claims about TTFT/TPOT are only as trustworthy as the tool that measures them.

3 System Architecture

Figure 1 shows the layering. The harness speaks OpenAI to each backend and emits SSE events through a pure `parse_sse_stream` function; metrics flow into the statistical harness, which in turn feeds the eight test suites.

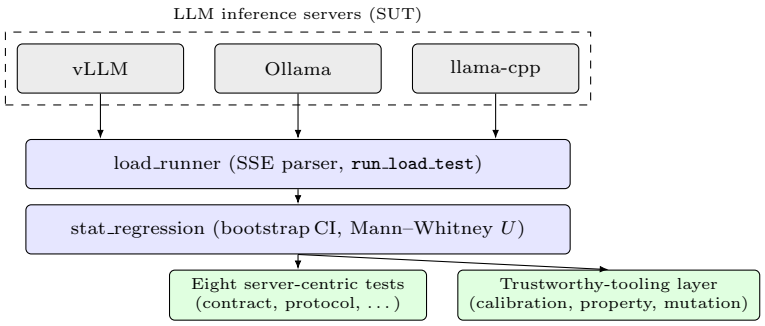

Figure 1: Harness architecture. The statistical harness feeds both the server-centric test suite and the trustworthy-tooling layer.

4 Server-Centric Test Methodology

Table 1 maps each test to a named software-testing methodology and to precedent from established LLM-serving projects.

Figure 2 positions the eight tests along two axes—whether they target structural or behavioural invariants, and whether they need a single backend or multiple.

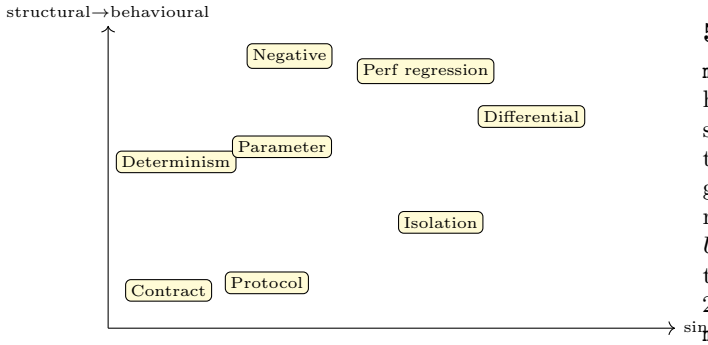

Figure 2: Coverage map for the server-centric test suite. Tests on the right-hand side consume multiple backends; tests higher on the y-axis exercise richer behavioural invariants.

5 Trustworthy-Tooling Layer

5.1 Measurement calibration

An in-process `aiohttp` server emits a deterministic SSE stream with configurable pre-first-chunk and inter-chunk delays (see Fig. 3). The harness is considered calibrated when TTFT and TPOT recovered from the stream equal the injected delays within ± 50 ms on the test host. Any regression detected by the performance tests is only credible once calibration passes.

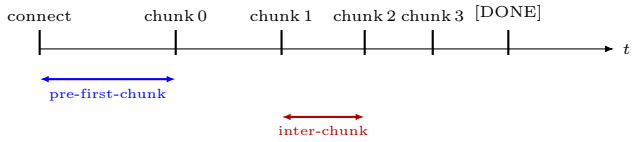

Figure 3: Calibration fixture. Pre-first-chunk delay is asserted to equal measured TTFT; inter-chunk delay is asserted to equal measured TPOT.

5.2 Property-based testing

The SSE parser is tested with Hypothesis [4] using a composite strategy that generates arbitrary printable payloads and then slices the resulting byte stream at arbitrary positions. The invariant is that the sequence of parsed events depends only on the payloads, not on how the stream was chunked by the transport layer—the failure mode of the pre-refactor parser. Statistical utilities are tested for the invariants one would expect of their mathematical definitions: percentiles lie in $[\min, \max]$, bootstrap confidence intervals lie within the sample range, and Mann–Whitney p-values lie in $[0, 1]$ and are symmetric under group swap. Across the eight Hypothesis tests the engine draws up to 1 050 randomised cases per full suite invocation (the sum of the per-test `max_examples` budgets).

5.3 Mutation testing

`mutmut` [5] generates 281 mutants of the statistical-regression harness (`src/stat_regression.py`). Before targeted tests the score was 201/281 (72%) killed. Pinning reference values for the percentile interpolation branch, exercising the default arguments of `bootstrap_ci`, and adding three Mann–Whitney reference computations—including a tie-handling case whose U and tie-correction term were derived by hand—raised the score to 247 killed, 29 survived, and 5 timeouts out of 281 (**88%** kill rate). The surviving 29 are concentrated in `mann_whitney_u` (11) and `bootstrap_ci` (9) and are dominated by equivalent mutants (e.g. encoding-string case variants that specify the same codec) and pure message-content changes that our regex-matched `ValueError` assertions do not distinguish from the originals.

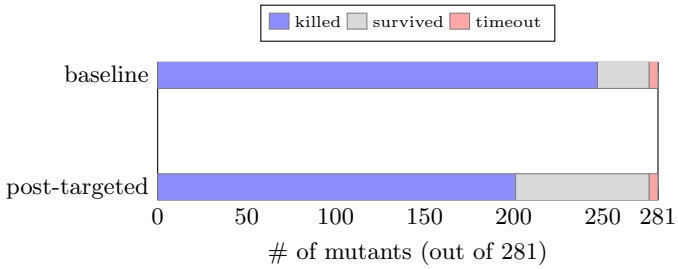

Figure 4: Mutation outcomes for `src/stat_regression.py` before and after the targeted test pass. The kill rate rose from 201/281 (72%) to 247/281 (88%); the 29 remaining survivors are distributed as `mann_whitney_u` (11), `bootstrap_ci` (9), `load_baseline` (4), `save_baseline` (3), and `percentile` (2).

6 Experimental Setup

Experiments use dual NVIDIA RTX 5070 GPUs with Qwen2.5-3B-Instruct on vLLM [1], Llama 3.2 Q4_K_M on Ollama [3], and Llama-3.2-3B-Instruct Q4_K_M on llama-cpp-python, a Python binding over llama.cpp [2]. All scenarios are driven from a single YAML configuration format to keep runs reproducible across backends.

7 Results

7.1 Performance

Table 2 summarises performance at five concurrent users. vLLM leads on every metric, driven by continuous batching and PagedAttention [1]. These numbers are reported as a baseline; the contribution of this paper is the surrounding apparatus that makes such numbers defensible as software-engineering artifacts.

Table 1: Eight server-centric tests layered on the existing harness.

Test	Methodology	Precedent	Assertion
Contract conformance	Schema/contract testing	llama.cpp OpenAI-compat server tests	Every response validates against the OpenAI chat-completion envelope schema.
Streaming protocol compliance	Protocol compliance	llama.cpp streaming tests	First chunk carries a role delta; `id` is stable across chunks; `finish_reason` and `[DONE]` appear exactly once.
Parameter honoring	Metamorphic testing	vLLM sampler tests (e.g. `test_ignore_eos.py`)	`max_tokens=N` $\Rightarrow$ `completion_tokens` $\leq N$; `stop=[...]` $\Rightarrow$ text ends before stop.
Determinism at T=0	Idempotency/metamorphic	vLLM determinism test suite	Identical prompts at T=0 yield identical output; rate bounded by a 95% bootstrap CI.
Statistical perf regression	Hypothesis testing	TGI fixed-request benchmark [6]	New p95 TTFT vs. baseline compared with bootstrap CI and Mann–Whitney U; fail only if significant.
Differential backends	N-version testing	Cross-backend reference comparison	Schema-validity rate across vLLM, llama.cpp, and Ollama agrees within ±5 percentage points.
Concurrency isolation	Invariant testing	llama.cpp parallel tests	Under N concurrent prompts with unique markers, each response contains only its own marker.
Negative/error path	Error-path testing	llama.cpp authentication and error-handling tests	Invalid bearer $\Rightarrow$ 401; malformed JSON $\Rightarrow$ 4xx (not 5xx or hang); over-context $\Rightarrow$ 4xx.

Table 2: Performance at 5 concurrent users.

Engine	Latency (ms)	TTFT (ms)	Throughput (tok/s)	JSON (%)
vLLM	**260**	**260**	**61.5**	100
Ollama	1426	1426	12.1	100
llama-cpp-python	2343	2343	6.0	100

7.2 Test suite expansion

Table 3 lists the test suites added by this revision and their current pass counts. The unit-level invocation (`pytest -m "not integration"`) reports 184 passing tests in ∼13 s; the integration suite adds 11 tests that are deselected when no local backend is reachable.

Table 3: New test suites and pass counts (unit mode).

Suite	Tests	Role
`test_contract.py`	8	Contract #1
`test_streaming_protocol.py`	6	Protocol #2
`test_param_honoring.py`	9	Parameter #3
`test_determinism.py`	2	Determinism #4
`test_stat_regression.py`	5	Perf regression #5
`test_differential.py`	3	Differential #6
`test_concurrency_isolation.py`	2	Isolation #7
`test_negative.py`	4	Negative #8
`test_properties.py`	8	Property-based (Hypothesis)
`test_calibration.py`	3	Calibration
`test_stat_harness.py`	31	Statistical math pins
All pre-existing suites	103	baseline
Total (unit mode)	**184**	—

7.3 Code coverage

Table 4 gives the line coverage reported by `pytest-cov`. The statistical harness is completely covered; the load runner is at 94%. Uncovered lines in `llm_tester.py` and `niah_runner.py` are the argparse entry points and the needle-in-a-haystack scenario, exercised only by integration runs.

Table 4: Line coverage by source module (unit mode).

Module	Stmts	Miss	Cov %
`stat_regression.py`	88	0	**100**
`comparator.py`	23	0	100
`ramp_runner.py`	37	0	100
`metrics.py`	78	2	97
`reporter.py`	78	4	95
`load_runner.py`	196	11	94
`config_loader.py`	52	3	94
`consistency.py`	55	5	91
`validator.py`	23	2	91
`html_reporter.py`	98	24	76
`niah_runner.py`	44	44	0
`llm_tester.py`	127	127	0
Total	**899**	**222**	**75**

7.4 Calibration

`test_calibration.py` spins up an in-process `aiohttp` server and runs three scenarios: a 150 ms pre-first-chunk delay (to target TTFT), an 80 ms inter-chunk delay (to target TPOT), and a composite end-to-end run (to assert latency is not under-reported). In each scenario the recovered metric must lie within ±50 ms of the injected delay; the tolerance is deliberately loose so the calibration check survives CI-host jitter without becoming flaky.

7.5 Mutation testing

The run yields 247 killed, 29 survived, and 5 timeouts out of 281 mutations. All five timeouts occur in `mann_whitney_u`, consistent with mutations that induce an infinite loop in its ranking code; they are reported separately from the 247 kills.

8 Discussion

The methodology matters more than any individual number. A naive single-run comparison reduces the vLLM advantage to a scalar; under this harness the same advantage is an effect whose significance is quantified by a Mann–Whitney U test and whose measurement provenance is defended by the calibration and property layers. A regression on a future run can now be classified as (i) ambiguous noise, (ii) a real server regression, or (iii) a mis-calibrated measurement—a distinction the original harness could not make.

Three observations:

- Determinism at T=0 is not free. Backends that reorder floating-point reductions under concurrency can produce different output for identical inputs, and an equality-based metamorphic test catches this without relying on any assumption about the reduction order inside the backend.
- Concurrency isolation is the cheapest test with the highest bug-catching signal-to-noise on this harness. Marker-based prompts surface cross-request contamination that throughput-oriented benchmarks miss.
- The 16 percentage-point lift in mutation score was produced by a single small test class of ten targeted tests—worth recording as a cheap ratio for mutation-testing planning on statistical code.

9 Harness Internals

- **SSE parser.** A pure `parse_sse_stream` helper—taking an iterable of byte chunks and yielding `SSEEvent` objects—makes the parser independent of the HTTP transport and drivable by property-based tests. Hypothesis exposed a subtlety where Python's `str.splitlines` treats `U+001E` (record separator) and similar control characters as line boundaries while the SSE grammar does not; the test strategy's alphabet was tightened accordingly.
- **Statistical regression harness.** Percentile, percentile-method bootstrap CI, and continuity-corrected Mann–Whitney U with tie correction, implemented against the standard library alone; no `scipy` dependency.
- **OpenAI schemas.** Draft-07 schemas for both the chat-completion envelope and the streaming chat-completion-chunk envelope, used by the contract and protocol tests.
- **Parameter plumbing.** `max_tokens` and `stop` are threaded through the load runner so the metamorphic tests can vary them without patching internal state.
- **Baseline persistence.** `save_baseline`/ `load_baseline` store per-scenario JSON reports so statistical regressions can be compared across runs.

10 Limitations

The test suite is methodology-complete but not exhaustive: determinism is checked only at T=0; the differential suite uses three backends, not the full matrix; the mutation-testing target is currently only the statistical harness, not the load runner; and integration-mode runs depend on a backend listening at `localhost:8080`. Hardware and model selection are fixed and were not themselves varied.

11 Conclusion

A benchmark can report *which backend is faster*; the harder question for a software-testing course is *why we should believe the answer.* By layering eight named testing methodologies onto a single harness, and by testing the harness itself against calibration, property, and mutation baselines, this study produces claims that are defensible as software-engineering artifacts rather than as single-run measurements. Under the evaluated conditions vLLM remains the best performer; the added machinery ensures that if that verdict ever changes, the change will be attributable and quantified.

12 Future Work

Future enhancements include: extending mutation testing to `load_runner.py`; adding P99 tail-latency regression tests; expanding the differential suite to the full model matrix; running the calibration harness periodically in CI to detect host drift; and integrating a GitHub Actions workflow that executes the unit suite on every PR and the integration suite nightly against a live backend.

References

[1] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., and Stoica, I. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pp. 611–626, 2023.

[2] Gerganov, G., and contributors. *llama.cpp: LLM inference in C/C++.* Open-source project. https://github.com/ggerganov/llama.cpp. Accessed April 2026.

[3] Ollama project. *Ollama: Get up and running with large language models locally.* Open-source project. https://github.com/ollama/ollama. Accessed April 2026.

[4] MacIver, D.R., and Hatfield-Dodds, Z. *Hypothesis: A new approach to property-based testing.* Journal of Open Source Software, 4(43):1891, 2019.

[5] Hovmöller, A. *mutmut: a mutation testing system for Python.* Open-source project. https://github.com/boxed/mutmut. Accessed April 2026.

[6] Hugging Face. *Text Generation Inference (TGI): a Rust, Python and gRPC server for text-generation inference.* Open-source project. https://github.com/huggingface/text-generation-inference. Accessed April 2026.